

INTEGRANTES

- Matheus Pastore
- Felipe Yuki
- Davi Conde

DESCRICAO GERAL DO PRODUTO

O UniHub e uma plataforma web para centralizar a vida academica e social de estudantes universitarios. O sistema reúne autenticação, perfil, dashboard, eventos, agenda pessoal, forum academico, moradias, mensagens, notificacoes e painel administrativo.

Principais funcionalidades:

- Cadastro, login, logout e sessao com Flask-Login.
- Perfil do usuario com edicao de dados.
- Dashboard com resumo de eventos, topicos, moradias e notificacoes.
- Forum com topicos, respostas, avisos oficiais e moderacao.
- Eventos com criacao por moderadores/admins, edicao, detalhes e agenda pessoal.
- Moradias com cadastro, edicao, detalhes e controle de status.
- Mensagens entre usuarios e notificacoes.
- Painel admin para gerenciamento de usuarios, eventos e forum.

ARQUITETURA ADOTADA

O projeto usa Flask com o padrao Application Factory. A funcao 'create_app()' em 'app.py' centraliza a criacao da aplicacao, carrega configuracoes, inicializa extensoes, registra models, login, CSRF, blueprints, Swagger e comandos de seed.

Trecho principal:

```
def create_app(test_config=None):
    flask_app = Flask(__name__, template_folder="templates", static_folder="static")
    init_config(flask_app, test_config)
    init_db(flask_app)
    register_models()
    init_login(flask_app)
    init_wtf(flask_app)
    init_views(flask_app)
    init_swagger(flask_app)
    register_seed_command(flask_app)
    return flask_app
```

A estrutura separa responsabilidades por pastas:

- 'unihub/models': entidades SQLAlchemy.

- 'unihub/views': rotas e blueprints por dominio.
- 'unihub/forms': formularios Flask-WTF/WTFForms.
- 'unihub/ext': inicializacao de extensoes.
- 'unihub/services': seed e servicos de apoio.
- 'unihub/utlis': helpers de seguranca, autenticacao e respostas.
- 'templates': telas Jinja2 e componentes.
- 'tests': testes automatizados com 'unittest'.
- 'migrations': migrations Alembic/Flask-Migrate.

ORGANIZACAO EM MODULOS E ROTAS

As rotas sao organizadas em Blueprints:

- 'main': landing page, dashboard, perfil, configuracoes e healthcheck.
- 'auth': cadastro, login, logout e usuario autenticado.
- 'usuarios': consulta e atualizacao de usuario.
- 'forum': topicos, respostas e moderacao.
- 'eventos' e 'agenda': eventos e agenda pessoal.
- 'moradias': anuncios de moradia.
- 'mensagens': conversas entre usuarios.
- 'notificacoes': notificacoes do usuario.
- 'admin': telas administrativas.

O registro acontece em 'unihub/views/__init__.py', mantendo a inicializacao centralizada.

FORMULARIOS E VALIDACAO

O projeto usa Flask-WTF/WTFForms para validacao no servidor. Os formularios ficam em 'unihub/forms' e sao usados tanto para telas HTML quanto para contratos principais de entrada.

Exemplos:

- 'CadastroForm': nome, email, senha, confirmacao, curso, periodo, cidade e biografia.
- 'LoginForm': email e senha obrigatorios.
- 'EventoForm': titulo, descricao, categoria, data, horario e local.
- 'MoradiaForm': titulo, descricao, bairro, preco, vagas e caracteristicas.
- 'ForumTopicoForm' e 'ForumRespostaForm': validacao de topicos e respostas.

O CSRF e inicializado em 'unihub/ext/wtf.py'. Em testes ele e desativado para permitir testes automatizados sem token manual.

INTERFACE E TEMPLATES

A interface usa Jinja2, templates por modulo e componentes reutilizaveis. Existem layouts separados para paginas publicas, autenticacao e area logada. A camada visual usa Bootstrap e CSS proprio, com telas para landing page, login, cadastro, dashboard, perfil, notificacoes, forum, eventos, moradias, mensagens e administracao.

Arquivos importantes:

- 'templates/layouts/public.html'

- 'templates/layouts/auth.html'
- 'templates/layouts/app.html'
- 'templates/components/*'
- 'templates/auth/*'
- 'templates/eventos/*'
- 'templates/forum/*'
- 'templates/moradias/*'
- 'templates/main/*'

PERSISTENCIA E BANCO DE DADOS

O projeto usa SQLite em desenvolvimento/testes e SQLAlchemy como ORM. As migrations ficam em 'migrations/'.

Modelo ER resumido:

usuarios 1:N eventos
 usuarios 1:N forum_topicos
 usuarios 1:N forum_respostas
 usuarios 1:N moradias
 usuarios 1:N notificacoes
 usuarios 1:N agenda_eventos
 usuarios 1:N mensagens como remetente
 usuarios 1:N mensagens como destinatario

eventos 1:N agenda_eventos
 forum_topicos 1:N forum_respostas

Tabelas principais:

- 'usuarios': dados de autenticacao, perfil, role, selo e status.
- 'eventos': eventos universitarios e organizador.
- 'agenda_eventos': relacao unica entre usuario e evento salvo.
- 'forum_topicos': topicos e avisos oficiais.
- 'forum_respostas': respostas dos topicos.
- 'moradias': anuncios de moradia.
- 'mensagens': mensagens privadas entre usuarios.
- 'notificacoes': notificacoes internas.

Regras de integridade:

- 'usuarios.role': 'usuario', 'moderador' ou 'admin'.
- 'eventos.status': 'ativo', 'cancelado', 'encerrado' ou 'desativado'.
- 'moradias.status': 'disponivel', 'pausado', 'preenchido' ou 'desativado'.
- 'forum_topicos.status': 'aberto', 'resolvido', 'fechado' ou 'desativado'.
- 'forum_topicos.tipo': 'topico' ou 'aviso'.
- 'agenda_eventos': restricao unica para impedir salvar o mesmo evento duas vezes.

TESTES AUTOMATIZADOS

Os testes usam 'unittest', SQLite em memoria e seed automatico. A classe base cria a aplicacao em modo teste, sobe o banco, executa o seed e entrega um client Flask para os testes.

Cobertura principal:

- Autenticacao e cadastro.
- Rotas publicas e protegidas.
- Eventos e agenda.
- Forum e respostas.
- Moradias.
- Mensagens.
- Notificacoes.
- Usuarios e roles.
- Regras de banco e constraints.

Resultado validado no ambiente local:

Ran 91 tests in 36.491s
OK

TECNOLOGIAS UTILIZADAS

- Flask: framework web principal.
- Flask-Login: sessao e autenticacao.
- Flask-SQLAlchemy: ORM.
- Flask-Migrate/Alembic: versionamento do banco.
- Flask-WTF/WTForms: formularios e validacao no servidor.
- Jinja2: templates HTML.
- Bootstrap/CSS: interface responsiva.
- Flasgger: documentacao Swagger.
- python-dotenv: variaveis de ambiente.
- unittest: testes automatizados.
- SQLite: banco local e de testes.

COMO RODAR O PROJETO

Instalacao recomendada no Linux/macOS:

```
python3 -m venv venv  
source venv/bin/activate  
python setup_project.py
```

Instalacao recomendada no Windows PowerShell:

```
py -3 -m venv venv  
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.\\env\\Scripts\\Activate.ps1
```

```
python setup_project.py
```

Instalacao recomendada no Windows CMD:

```
py -3 -m venv venv  
venv\Scripts\activate.bat  
python setup_project.py
```

O script deve ser executado com a 'venv' ativa. Ele instala dependencias, gera arquivos '.env', aplica migrations e popula o banco inicial.

Instalacao manual no Linux/macOS:

```
python3 -m venv venv  
source venv/bin/activate  
python -m pip install --upgrade pip  
python -m pip install -r requirements.txt  
python -m pip install -e .  
python make_env.py  
flask --app app.py db upgrade  
flask --app app.py seed
```

Instalacao manual no Windows PowerShell:

```
py -3 -m venv venv  
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.\venv\Scripts\Activate.ps1  
python -m pip install --upgrade pip  
python -m pip install -r requirements.txt  
python -m pip install -e .  
python make_env.py  
flask --app app.py db upgrade  
flask --app app.py seed
```

Rodar no Linux/macOS:

```
flask --app app.py run --debug --port 5000
```

Rodar no Windows PowerShell:

```
flask --app app.py run --debug --port 5000
```

Com Invoke:

invoke setup
invoke dev

Testar:

python -m unittest

URLs principais:

- Landing page: 'http://127.0.0.1:5000/'
- Login: 'http://127.0.0.1:5000/auth/login'
- Dashboard: 'http://127.0.0.1:5000/dashboard'
- Swagger: 'http://127.0.0.1:5000/swagger/'

Usuario seed para demonstracao:

Admin: rafael.souza@uvv.br / senha123
Moderador: mariana.costa@uvv.br / senha123
Usuario: lucas.almeida@uvv.br / senha123

PRINCIPAIS DESAFIOS E SOLUCOES

- Modularizacao: o projeto foi dividido em models, views, forms, services, utils e ext para evitar rotas e regras misturadas em um unico arquivo.
- Controle de acesso: roles foram padronizadas em 'usuario', 'moderador' e 'admin', com helpers para proteger rotas sensiveis.
- Validacao: formularios Flask-WTF foram adicionados para validar entradas no servidor e melhorar a confiabilidade da aplicacao.
- Persistencia: SQLAlchemy e migrations garantem modelagem explicita, constraints e reproducibilidade do banco.
- Testes: a suite usa banco em memoria e seed automatico para testar fluxos principais sem depender do banco local.
- Entrega: '.env.example', README, migrations e seed foram mantidos para facilitar auditoria e execucao pelo professor.